

ÉCOLE NATIONALE SUPÉRIEURE D'ARTS ET MÉTIERS

ENSAM MEKNÈS — Maroc

Filière IATD SI — Intelligence Artificielle et Technologies des Données : Systèmes Industriels

RAPPORT DE TRAVAUX PRATIQUE 1

Manuel d'Exploration Unity VR

Prise en main de Unity 6 et Meta Quest

Séance 1 — Travaux Pratiques

Étudiant	DJERI-ALASSANI OUBENOUPOU
Étudiant	HINIMDOU MORSIA GUITDAM
Encadrants	Ajebli Ahmed Maalouf Imad
Module	Virtual and Augmented Reality
Filière	IATDSI — 4e Année
Date	Vendredi 11 Avril 2025

Introduction

Les interfaces classiques offrent de nombreux avantages en termes de simplicité d'utilisation et de rapidité de mise en œuvre. Cependant, elles présentent des limites majeures lorsqu'il s'agit de représenter des environnements complexes ou de permettre une interaction naturelle. L'absence d'immersion et le caractère souvent abstrait de ces interfaces peuvent réduire l'efficacité de l'utilisateur et limiter sa compréhension de certaines situations.

Face à ces contraintes, les technologies immersives telles que la réalité virtuelle (VR) et la réalité augmentée (AR) ont émergé comme des solutions innovantes. En plongeant l'utilisateur dans un environnement interactif, elles offrent une expérience plus intuitive, réaliste et engageante. Ces technologies sont aujourd'hui au cœur de nombreuses applications dans des domaines variés tels que l'industrie, la santé, la formation, la simulation ou encore le divertissement.

C'est dans ce contexte que s'inscrit ce rapport, qui documente de manière détaillée le déroulement de la première séance de travaux pratiques du module *Virtual and Augmented Reality*. L'objectif de cette séance est d'aller au-delà des concepts théoriques pour réaliser une prise en main concrète et complète du développement VR.

Pour ce faire, nous nous sommes appuyés sur **Unity 6**, l'un des moteurs de développement les plus utilisés au monde pour créer des expériences immersives, et sur le casque **Meta Quest**, reconnu pour son accessibilité et sa large diffusion. Maîtriser ces outils de développement est aujourd'hui une compétence essentielle pour un ingénieur en systèmes numériques. Ce travail nous a permis d'appréhender le fonctionnement global d'un système VR, de la configuration de l'environnement jusqu'à la mise en place d'interactions en immersion.

Afin de proposer une démarche pédagogique et technique rigoureuse, ce rapport n'a pas pour unique but d'expliquer *comment* réaliser chaque étape technique. Il s'attache surtout à détailler *pourquoi* chaque action est nécessaire, quel est son rôle précis dans l'architecture globale du projet, et quelles seraient les conséquences en cas d'omission.

Les principales réalisations de ce TP, abordées de manière progressive tout au long de ce document, se résument comme suit :

- **Mise en place** d'un environnement de développement complet et fonctionnel ;
- **Configuration** des outils nécessaires au développement en réalité virtuelle ;
- **Création et structuration** d'une scène 3D simple ;
- **Intégration** d'un système d'interaction avec les contrôleurs VR ;
- **Déploiement et test** de l'application directement sur le casque Meta Quest.

Objectifs du TP

À l'issue de cette séance, nous devons être capables de :

- Configurer un environnement de développement Unity 6 complet pour la VR.
- Créer un projet Unity 3D et le paramétrer pour cibler le casque Meta Quest.
- Construire une scène 3D simple comportant un sol, un éclairage et un objet interactif.
- Intégrer un joueur VR avec ses contrôleurs virtuels visibles dans la scène.
- Rendre un objet saisissable grâce aux contrôleurs physiques du casque.
- Compiler l'application et la déployer directement sur le casque Meta Quest.

Étape 1 — Installation de Unity Hub et Unity 6

1.1 Qu'est-ce que Unity Hub ?

Unity Hub est un gestionnaire centralisé pour toutes les versions de Unity. Il permet d'installer, de gérer et de lancer plusieurs versions de l'éditeur Unity simultanément, de gérer les projets et les licences, et de télécharger des modules complémentaires (support de plateformes cibles comme Android ou iOS).

Pourquoi utiliser Unity Hub ?

Sans Unity Hub, il serait difficile de gérer plusieurs versions de Unity, d'ajouter des modules de build, ou de s'authentifier. Unity Hub sert de portail unique pour accéder à tout l'écosystème Unity. Il est indispensable dans un contexte professionnel ou académique où plusieurs projets coexistent.

1.2 Installation de Unity 6

Depuis Unity Hub, on installe Unity 6, la version la plus récente et stable du moteur. Cette version apporte de nombreuses améliorations en termes de rendu, de performances sur mobile, et de support natif pour les plateformes XR (eXtended Reality).

Les trois modules indispensables

Lors de l'installation de Unity 6, il est impératif de cocher les trois modules suivants :

- **Android Build Support** : Ce module est la couche de base qui permet à Unity de compiler du code pour le système d'exploitation Android. Sans lui, Unity ne peut pas générer d'APK (Android Package), le format d'application pour Android et Meta Quest.
- **Android SDK & NDK Tools** : Le SDK (Software Development Kit) Android fournit les bibliothèques et outils nécessaires pour cibler des versions précises d'Android. Le NDK (Native Development Kit) permet la compilation de code C++ natif, utilisé pour les calculs bas niveau. Ces outils sont embarqués directement dans Unity pour éviter toute incompatibilité de version.
- **OpenJDK** : Java Development Kit est requis pour compiler les composants Java/Kotlin de l'application Android. Meta Quest tourne sur Android, donc une partie de l'application est écrite en Java en arrière-plan. Unity utilise OpenJDK, sa version intégrée, pour garantir la compatibilité.

Logiciel / Module	Rôle	Source
Unity Hub	Gestionnaire d'installations Unity	unity.com
Unity 6	Moteur de jeu / environnement XR	via Unity Hub
Android Build Support	Compilation pour Android/Quest	module Unity
Android SDK & NDK Tools	Outils bas niveau Android	module Unity
OpenJDK	Environnement Java pour le build	module Unity
OpenXR Plugin	Standard ouvert XR	Package Manager
XR Interaction Toolkit	Interactions VR	Package Manager
Meta Quest Link	Test en mode connecté	Meta

Remarque importante

Ces trois modules peuvent être ajoutés ultérieurement via Unity Hub (menu Installs > icône des trois points > Add Modules), mais il est fortement recommandé de les sélectionner dès le départ pour éviter de relancer une procédure d'installation longue. Leur absence rend l'export vers Meta Quest totalement impossible.

Étape 2 — Création du projet Unity

2.1 Choisir le bon template

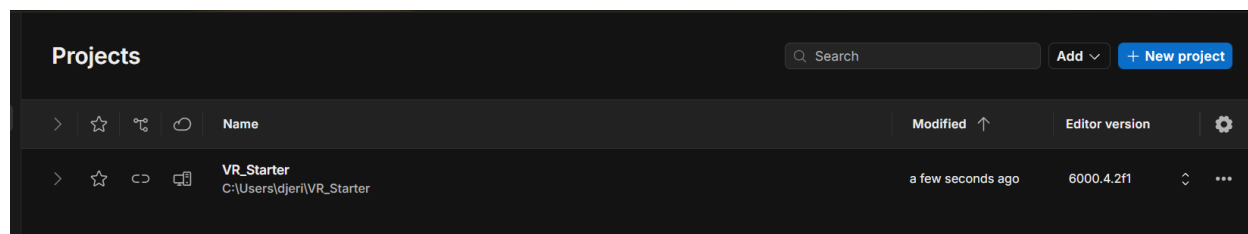
Dans Unity Hub, on crée un nouveau projet en sélectionnant le template 3D Core (ou 3D Built-in). Ce choix est stratégique : il s'agit du template le plus léger et généraliste, qui n'embarque aucun pipeline de rendu avancé par défaut. Cela nous donne un contrôle total sur la configuration et une base propre pour y ajouter uniquement ce dont on a besoin.

Pourquoi le template 3D Core et non URP ou HDRP ?

Le template Universal Render Pipeline (URP) est optimisé pour les plateformes mobiles mais nécessite une reconfiguration des shaders pour la VR. Le template HDRP (High Definition) est trop lourd pour Meta Quest. Pour une première séance exploratoire, le template 3D Core offre la meilleure compatibilité immédiate avec les packages XR que nous allons installer.

2.2 Nommer le projet

Le projet est nommé VR_Starter. Ce nom est plus qu'une simple convention : il reflète l'intention du projet (démarrateur VR) et facilitera l'identification dans Unity Hub. Il est recommandé d'éviter les espaces et les accents dans les noms de projet Unity pour prévenir des erreurs de compilation sous Android.



Étape 3 — Configuration de Unity pour la VR

3.1 Comprendre l'architecture XR de Unity

Unity utilise un système modulaire pour le support VR/AR, regroupé sous le terme XR (eXtended Reality). Ce système repose sur plusieurs couches :

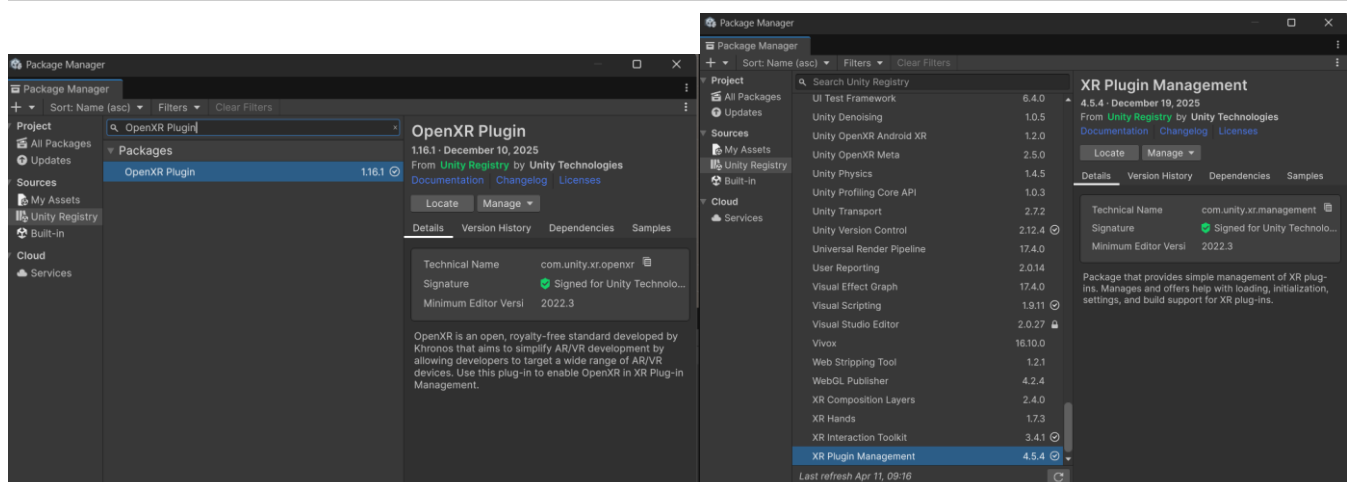
- **XR Plugin Management** : Le chef d'orchestre. C'est le gestionnaire central qui détecte et charge les plugins XR appropriés selon la plateforme cible. Il fournit une interface unifiée pour activer OpenXR ou d'autres runtimes.
- **OpenXR Plugin** : Le protocole standard. OpenXR est une norme ouverte, gérée par le consortium Khronos Group, qui définit une API commune pour tous les casques VR. Cela signifie qu'un projet configuré avec OpenXR peut théoriquement cibler Meta Quest, HTC Vive, Valve Index, etc., avec le même code. C'est le choix de l'industrie pour la portabilité.
- **XR Interaction Toolkit (XRI)** : La boîte à outils d'interactions. Ce package fournit des composants prêts à l'emploi pour les interactions VR : saisie d'objets, locomotion, raycasting, interfaces UI en 3D. Sans lui, il faudrait coder toute la logique d'interaction depuis zéro.

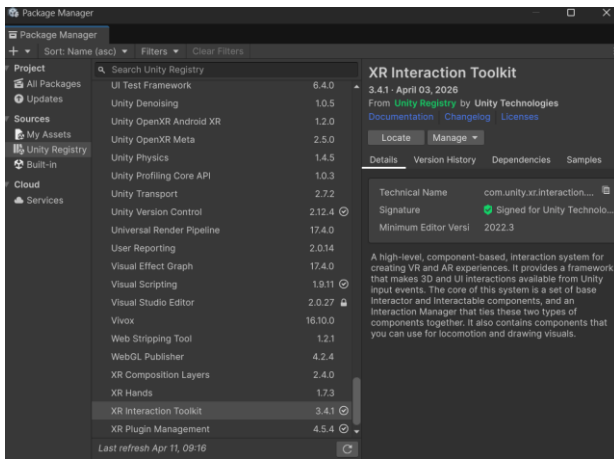
3.2 Installation via le Package Manager

L'installation se fait via Window > Package Manager en sélectionnant la vue Unity Registry (le registre officiel Unity). Cette approche garantit de télécharger les versions validées et compatibles entre elles, à la différence de packages tiers qui pourraient introduire des conflits.

Starter Assets du XR Interaction Toolkit

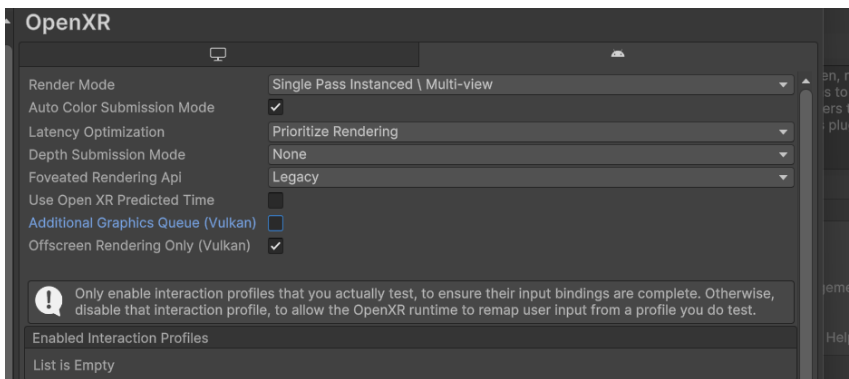
Il est essentiel d'importer les Starter Assets depuis l'onglet Samples du XRI. Ces assets contiennent des prefabs préconfigurés (comme le XR Origin) qui économisent un temps considérable de configuration manuelle. Sans eux, la mise en place d'un joueur VR fonctionnel demanderait des dizaines d'étapes supplémentaires.





3.3 Activation d'OpenXR

Dans Edit > Project Settings > XR Plug-in Management, on sélectionne l'onglet Android (représenté par l'icône Meta Quest) et on coche OpenXR. Cette action déclare à Unity que, lorsque l'application tournera sur Meta Quest, elle devra communiquer avec le casque via le protocole OpenXR.



3.4 Profils de contrôleurs Meta Quest

Dans les réglages OpenXR, il faut ajouter les profils de contrôleurs Meta Quest Touch / Touch Pro / Touch Plus. Ces profils définissent le mappage des boutons et des capteurs des contrôleurs physiques vers des entrées Unity. Sans eux, Unity ne saurait pas comment interpréter les signaux envoyés par les manettes du casque.

Pourquoi plusieurs profils ?

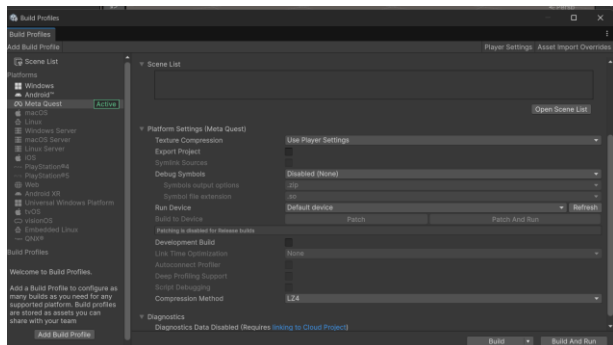
Meta Quest a commercialisé plusieurs générations de contrôleurs (Touch v1, Touch Pro, Touch Plus). En activant plusieurs profils, on garantit la compatibilité avec différents modèles de casques. Unity sélectionne automatiquement le bon profil au démarrage selon le casque détecté.

Étape 4 — Changement de plateforme cible

4.1 Pourquoi changer de plateforme ?

Par défaut, Unity cible la plateforme PC (Windows/Mac/Linux). Or, Meta Quest est un appareil Android. Il est donc indispensable de dire à Unity de compiler le projet pour Android afin de générer une application compatible avec le casque.

Cette opération se réalise via File > Build Profiles (Unity 6) ou File > Build Settings (versions antérieures), puis en sélectionnant la plateforme Android / Meta Quest et en cliquant sur Switch Platform.



4.2 Ce que fait concrètement Switch Platform

Ce bouton déclenche une série d'opérations internes dans Unity :

1. Réimportation de tous les assets selon les paramètres de compression Android (textures, sons, shaders).
2. Reconfiguration du moteur de script pour cibler l'architecture ARM (celle des processeurs Snapdragon de Meta Quest).
3. Activation des outils de build Android (SDK, NDK, OpenJDK installés à l'étape 1).
4. Génération d'un nouveau cache d'assets optimisé pour la nouvelle plateforme.

Durée de l'opération

Cette opération prend plusieurs minutes la première fois car Unity doit réimporter et recompresser tous les assets du projet. Les fois suivantes, seuls les assets modifiés sont retraités, ce qui est beaucoup plus rapide. Il est donc conseillé de changer de plateforme tôt dans le projet pour éviter des attentes longues à un stade avancé.

Étape 5 — Construction de la scène 3D

5.1 Le rôle de la scène dans Unity

Une scène Unity est l'espace dans lequel se déroule l'expérience VR. Elle contient tous les objets 3D (GameObjects), les lumières, les caméras, et les scripts. En VR, la scène représente l'environnement virtuel que l'utilisateur va percevoir comme réel grâce au casque.

5.2 Le sol (Plane)

On ajoute un objet Plane (Hiérarchie > clic droit > 3D Object > Plane), positionné à X=0, Y=0, Z=0. Cet objet remplit plusieurs fonctions essentielles :

- Référentiel spatial : il donne à l'utilisateur une surface de référence qui ancre l'expérience dans un espace compréhensible. Sans sol, l'utilisateur se sent « flotter » dans le vide, ce qui génère une sensation d'inconfort appelée cybersickness.
- Surface de collision : le Plane possède nativement un Mesh Collider, ce qui empêche les objets physiques (comme le cube) de traverser le sol.
- Repère visuel : en VR, les repères visuels sont essentiels pour maintenir l'équilibre cognitif de l'utilisateur.

5.3 Le cube interactif

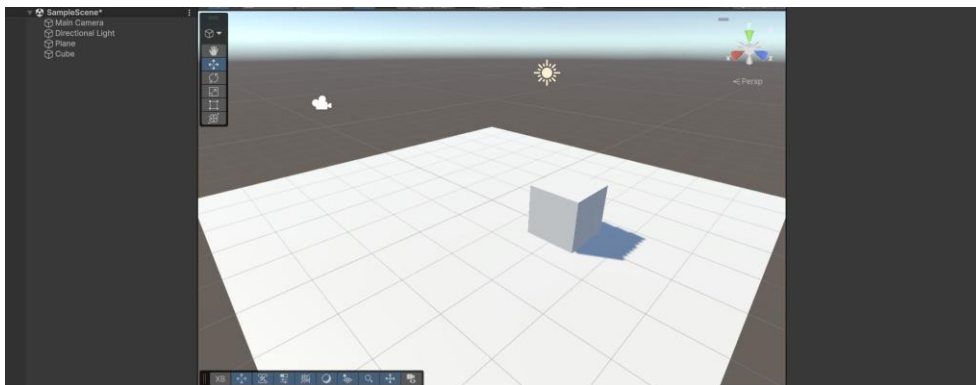
On ajoute un Cube (3D Object > Cube) positionné à X=0, Y=0.5, Z=2. La position Y=0.5 est calculée pour que la base du cube repose exactement sur le sol (un cube Unity a une taille par défaut de 1 unité = 1 mètre, donc son centre est à 0.5 mètre du sol). La position Z=2 le place à 2 mètres devant l'utilisateur, soit à portée de main dans la VR.

Pourquoi Z=2 précisément ?

En VR, l'espace d'interaction optimal se situe entre 0.5 m et 1.5 m devant l'utilisateur. Cependant, Z=2 permet de voir clairement l'objet en entrant dans la scène, avant d'ajuster la position des contrôleurs. C'est un positionnement pédagogique pour cette première séance.

5.4 L'éclairage (Directional Light)

La Directional Light est la lumière principale de la scène. Elle simule la lumière du soleil : une source lumineuse infiniment distante qui éclaire tous les objets dans la même direction. Sans lumière, tous les objets apparaissent noirs dans le casque. La Directional Light est présente par défaut dans les nouvelles scènes Unity, mais il est important de la vérifier car une scène sans lumière est inutilisable en VR.



Étape 6 — Ajout du joueur VR (XR Origin)

6.1 Qu'est-ce que le XR Origin ?

Le XR Origin est le composant central qui représente l'utilisateur dans la scène VR. C'est un Game Object hiérarchique qui contient trois sous-éléments essentiels :

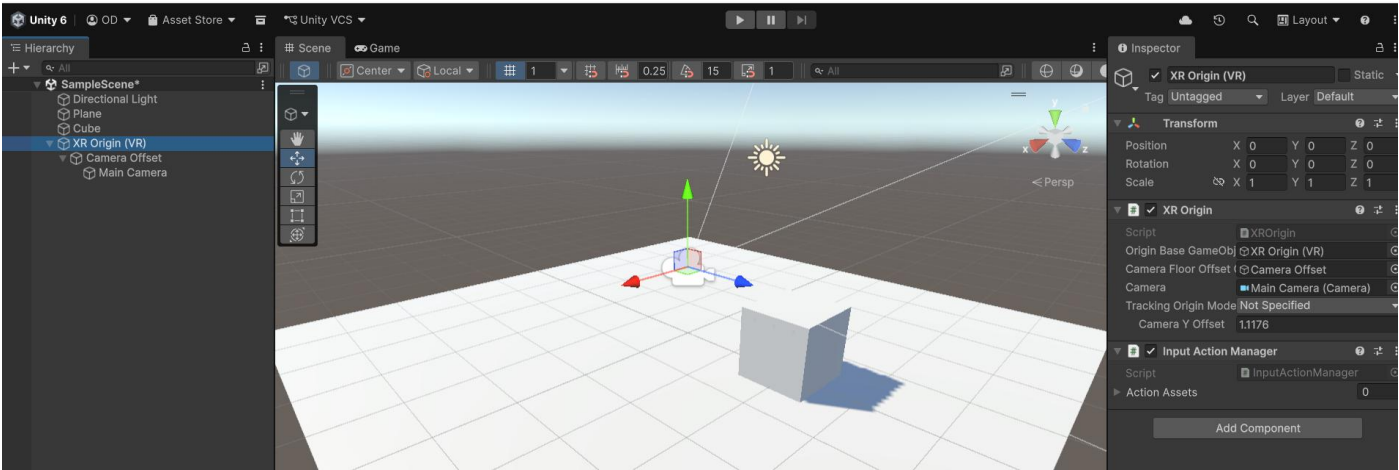
Composant	Rôle
Main Camera	Représente les yeux de l'utilisateur. Ses mouvements sont trackés directement depuis les capteurs IMU du casque. Le rendu stéréoscopique (deux images légèrement décalées pour créer la profondeur 3D) est géré automatiquement.
Left Controller	Représente le contrôleur gauche. Sa position et son orientation dans l'espace virtuel sont synchronisées en temps réel avec la position de la manette gauche physique, grâce au tracking inside-out du Meta Quest.
Right Controller	Représente le contrôleur droit. Même principe que le contrôleur gauche. C'est via ces deux contrôleurs que l'utilisateur interagit avec les objets de la scène.

6.2 Positionnement du XR Origin

Le XR Origin est placé à X=0, Y=0, Z=0, exactement à l'origine de la scène. Ce positionnement est fondamental : il définit le point de départ de l'utilisateur dans l'espace VR. Le cube placé en Z=2 sera donc bien visible devant lui dès le lancement de l'application.

Tracking inside-out

Meta Quest utilise des caméras embarquées sur le casque pour traquer la position de la tête et des contrôleurs sans base-station externe. Cette technologie s'appelle inside-out tracking. Unity, via OpenXR, reçoit en temps réel les coordonnées calculées par le casque et les applique automatiquement à la Main Camera et aux contrôleurs du XR Origin.



Étape 7 — Rendre le cube interactif

7.1 Composant Rigidbody

On sélectionne le Cube dans la Hierarchy et on lui ajoute un composant Rigidbody depuis l'Inspector. Le Rigidbody est le composant physique de Unity : il soumet l'objet au moteur de physique, ce qui implique :

- Gravité : l'objet tombe si rien ne le soutient.
- Collisions : l'objet réagit aux chocs avec d'autres colliders (sol, murs, autres objets).
- Forces et couples : l'objet peut être propulsé, lancé, faire pivoter de manière réaliste.

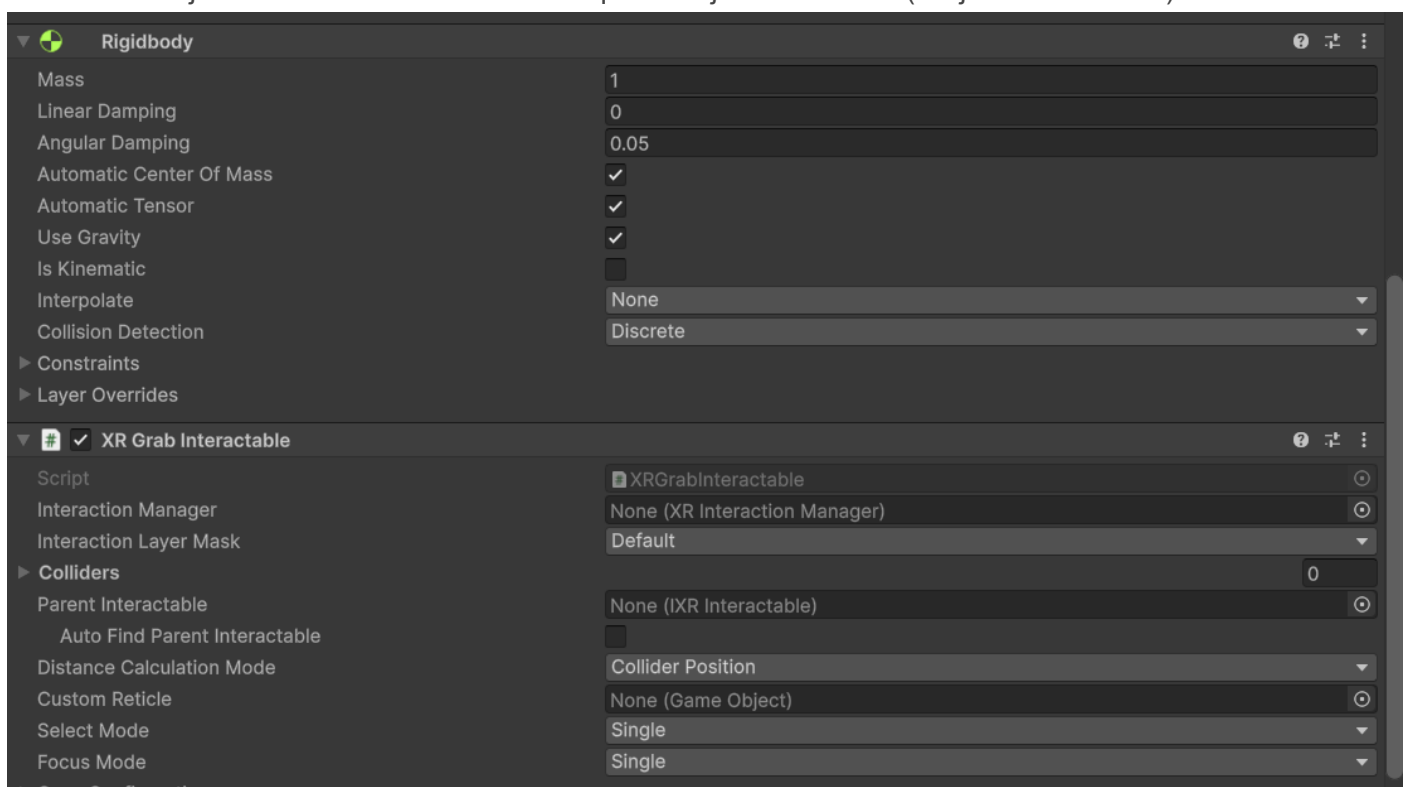
Sans Rigidbody, le cube serait un simple mesh statique : visible, mais ne pouvant pas être affecté par la physique ni lancé de manière réaliste lors de la saisie VR.

7.2 Composant XR Grab Interactable

Le composant XR Grab Interactable, issu du XR Interaction Toolkit, est le composant qui rend l'objet saisissable avec les contrôleurs VR. Il agit comme un pont entre le système d'interaction XR (les contrôleurs) et l'objet 3D.

Concrètement, quand l'utilisateur approche sa manette de l'objet et appuie sur la gâchette (Grip), le XR Grab Interactable :

5. Détecte la tentative de saisie via le XR Interaction Manager.
6. Attache l'objet au contrôleur (l'objet suit le mouvement de la manette).
7. Libère l'objet avec le bon vecteur vitesse quand le joueur relâche (l'objet est « lancé »).



Rôle du Collider

Un Cube Unity possède nativement un Box Collider. Ce collider est essentiel pour deux raisons : d'abord, le moteur de physique l'utilise pour calculer les collisions ; ensuite, le XR Interaction Toolkit l'utilise pour détecter la proximité entre le contrôleur et l'objet. Sans collider, la saisie VR serait impossible.

Étape 8 — Connexion du casque

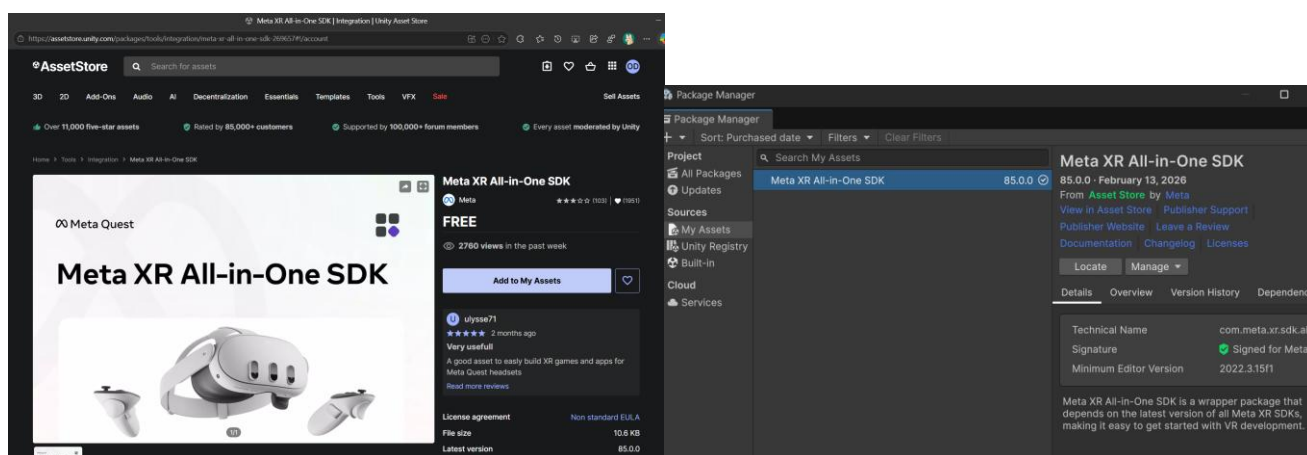
8.1 Connexion physique par USB

On connecte le Meta Quest au PC via un câble USB-C. Cette connexion permet deux choses : premièrement, le transfert de l'application compilée (APK) vers le casque ; deuxièmement, si Meta Quest Link est installé, la possibilité de tester directement depuis l'éditeur Unity sans compiler.

8.2 Autorisations et mode développeur

Lors de la première connexion, le casque affiche une invite demandant d'autoriser l'accès depuis le PC. Il faut obligatoirement accepter, sinon le PC ne peut pas communiquer avec le système de fichiers du casque.

L'activation du mode développeur (via l'application Meta pour smartphone) est indispensable car elle désactive les restrictions de sécurité qui empêchent l'installation d'applications non publiées sur le Meta Quest Store. Sans ce mode, seules les applications officielles peuvent être installées.



Meta XR All-in-One SDK

Si Unity ne détecte pas le casque après connexion, il est recommandé d'installer le Meta XR All-in-One SDK depuis l'Unity Asset Store. Ce SDK regroupe l'ensemble des outils Meta et inclut les pilotes de communication USB nécessaires. Il est disponible gratuitement.

Étape 9 — Paramétrage du Build Meta Quest

9.1 Identité de l'application Android

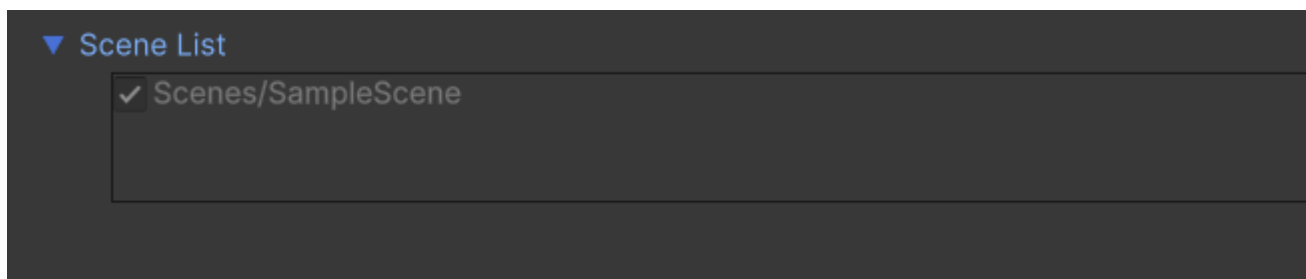
Dans Player Settings (Edit > Project Settings > Player), on renseigne les informations d'identité de l'application :

- **Nom du produit** : Le nom affiché dans la bibliothèque du casque (ex. : VR_Starter). Ce nom doit être clair et court.
- **Nom de l'entreprise** : Le nom de l'organisation développeur (ex. : ENSAM). Il fait partie de l'identifiant unique.

- **Package Name** : Aussi appelé Bundle Identifier, il suit le format `com.organisation.nomprojet` (ex. : `com.ecole.vr_starter`). Cet identifiant est unique au niveau mondial et permet au système Android d'identifier l'application parmi toutes les autres installées.

9.2 Ajout de la scène au build

Dans la fenêtre Build Profiles, il faut cliquer sur Add Open Scenes pour inclure la scène courante dans la liste des scènes à compiler. Unity ne compile que les scènes explicitement listées. Si cette étape est omise, l'application est compilée sans aucune scène, et le casque ne verra qu'un écran noir.



Étape 10 — Compilation et Déploiement

10.1 Le processus Build and Run

En cliquant sur Build and Run, Unity déclenche l'ensemble de la chaîne de compilation :

8. Compilation des scripts C# en bytecode IL2CPP pour Android ARM.
9. Empaquetage des assets (textures compressées ETC2/ASTC, sons, scènes).
10. Génération de l'APK (Android Package), le fichier d'installation de l'application.
11. Transfert de l'APK vers le casque via ADB (Android Debug Bridge).
12. Installation et lancement automatique de l'application sur le casque.

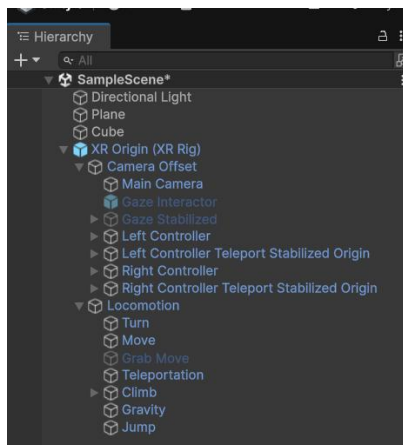
10.2 Résultat attendu dans le casque

Une fois l'application lancée, l'utilisateur doit percevoir :

- Un sol gris (le Plane) sous ses pieds, donnant un ancrage visuel à l'espace.
- Un cube blanc placé à environ 2 mètres devant lui, à hauteur de main.
- Ses contrôleurs virtuels visibles, dont la position reflète celle de ses manettes physiques.
- La possibilité de saisir le cube en approchant un contrôleur et en appuyant sur la gâchette.
- Le cube qui tombe sous l'effet de la gravité lorsqu'il est lâché.

Structure de la scène Unity

La hiérarchie finale de la scène reflète l'organisation logique de l'expérience VR :



GameObject	Rôle dans la scène
XR Origin	Racine du joueur VR. Contient la caméra et les contrôleurs.
↳ Main Camera	Rendu stéréoscopique, suivi de la tête (Head Tracking).
↳ Left Controller	Représentation et input du contrôleur gauche.
↳ Right Controller	Représentation et input du contrôleur droit.
Directional Light	Éclairage principal de la scène (simulation soleil).
Plane	Sol de l'environnement VR, surface de collision.
Cube	Objet interactif, saisissable via XR Grab Interactable.

Problèmes courants et solutions

Problème observé	Cause probable & Solution
Build impossible vers Meta Quest	Modules Android Build Support, SDK/NDK ou OpenJDK manquants. Solution : les ajouter via Unity Hub > Installs > Add Modules.
Unity ne détecte pas le casque	Mode développeur non activé ou autorisations USB refusées. Solution : activer le mode développeur et accepter l'invite USB dans le casque.
Écran noir dans le casque	La scène n'a pas été ajoutée au build. Solution : Add Open Scenes dans la fenêtre Build Profiles.
Le cube ne peut pas être saisi	Composant XR Grab Interactable ou XR Interaction Manager manquant. Vérifier que le XR Origin contient bien les interactors de contrôleurs.
Objets sombres ou noirs	Absence de Directional Light. Solution : ajouter une lumière directionnelle via GameObject > Light > Directional Light.

Conclusion

Ce travail pratique constitue une introduction complète et structurée au développement en réalité virtuelle (VR) à l'aide de Unity 6 et du casque Meta Quest. En parcourant les différentes étapes du processus de création, nous avons pu mettre en place la totalité de la chaîne de développement : depuis l'installation de l'environnement jusqu'au déploiement d'une application fonctionnelle sur un dispositif réel.

Au cours de ce projet, nous avons acquis des compétences techniques essentielles en justifiant chaque étape par son utilité concrète :

- ✓ Unity Hub pour la gestion de l'environnement de travail.
- ✓ Les modules Android pour assurer la compilation cross-platform.
- ✓ Le standard OpenXR pour garantir la portabilité de l'application.

L'utilisation du XR Interaction Toolkit nous a permis de comprendre comment les mouvements de l'utilisateur sont traduits dans l'espace 3D. La mise en place de la scène et l'ajout d'un objet interactif manipulable ont illustré de manière concrète les principes fondamentaux de l'interaction en VR.

L'intégration synergique de composants tels que le XR Origin, le Rigidbody pour la gestion de la physique, et le XR Grab Interactable pour la saisie, a permis d'obtenir une expérience immersive, fluide et intuitive.

Les résultats obtenus sont très satisfaisants : l'application s'exécute correctement sur le casque Meta Quest, offrant une bonne qualité d'affichage et une interaction précise avec les contrôleurs.

Cette première expérience pose des bases solides pour la réalisation d'applications plus poussées. En perspective, il sera possible d'enrichir ce travail lors des prochaines séances en y intégrant des fonctionnalités avancées telles que la locomotion immersive, des interfaces utilisateur (UI) en 3D, des interactions complexes multi-objets, ainsi que l'intégration de contenus visuels de haute qualité. À plus long terme, l'ajout de systèmes intelligents basés sur l'intelligence artificielle permettra d'ouvrir la voie à des applications concrètes dans divers domaines professionnels.

Compétences acquises

À l'issue de ce TP : configuration d'un pipeline de build Android/VR dans Unity, compréhension de l'architecture XR (Plugin Management, OpenXR, XRI), création d'une scène 3D interactive, intégration d'un joueur VR complet, déploiement d'une application sur Meta Quest.